

工具调用 折叠机制对比

Pi Stuff 当前实现、Claude Code 2.1.220 行为与公开镜像源码

官方二进制回答 Claude 实际做了什么；固定源码快照解释它可能如何做到。两类证据分开标注，再与 Pi Stuff 的可审计实现比较。

目录

本报告比较的是分组状态机，不是几行终端文案。官方二进制行为、非官方源码快照与 Pi Stuff 当前源码分层取证，最后收束为可进入设计评审的演进边界。

01 · 执行摘要

02 · 证据与比较口径

03 · Pi Stuff 当前实现

04 · Claude Code 2.1.220 行为

05 · 公开镜像揭示的实现链路

06 · 差异不是样式，而是风险模型

07 · 建议的演进路径

证据基准: Pi Stuff cf12251; Claude 官方二进制 2.1.220; 非官方镜像 6f6f12。镜像的来源、完整性与产品版本均未获 Anthropic 认证。

执行摘要

两套产品都把折叠限定在显示层，但优化目标不同。**Pi Stuff** 优先保证错误和副作用可见，**Claude Code** 优先维持连续而安静的探索投影。公开镜像进一步解释了这种差异如何由分类、归并和渲染三个环节共同形成。

2+	1	700 ms
Pi Stuff 最小分组调用数	Claude 最小折叠调用数	镜像中的目标最短显示时间

五个结论

- **Pi Stuff** 更保守：至少两个相邻探索调用、同一 assistant message、全部成功后才折叠。
- **Claude Code 2.1.220** 更激进：单次调用也折叠，并可跨多个 assistant API message 聚合。
- 公开镜像与黑盒行为在最小门槛、连续窗口、唯一文件计数、失败保留和双态文案上相互印证。
- 源码还揭示了 Tool-owned 分类、显式 MCP 白名单、目标防抖，以及经典终端与全屏路径的差异。
- 推荐借鉴语义摘要和连续探索段，但保留两项门槛、失败展开与完整详情。

建议如何读这份报告

第 3、4 章分别还原两套状态机；第 5 章把差异映射到用户风险；第 6 章给出一条不破坏当前安全边界的迁移顺序。若只做一个决策，应先同意以下原则：

DECISION

借 **Claude** 的信息架构，不借它的失败隐藏策略。扩大连续探索段以前，先把摘要从 Tool 名单改成读、搜、列等语义结果，并保留失败逐项显示。

证据与比较口径

官方 2.1.220 二进制仍是 Claude 当前行为的主证据；非官方公开镜像只用于解释可能的实现机制，并以固定提交引用，不能替代版本化官方源码。

三层证据，不混用结论

证据层	固定对象	可回答	不能回答
Pi 源码与 PTY	cf12251、Pi 0.83.0	当前实现与生命周期	Claude 内部设计
Claude 官方二进制	Linux x64 2.1.220	真实默认行为与视觉结果	内部函数和数据结构
非官方公开镜像	6f6f12	静态实现链路与潜在条件路径	真实性、完整性及对应产品版本

镜像来源限制

tanbiralam/claude-code 自称内容来自 2026-03-31 的 source map 暴露。它不是 Anthropic 官方仓库，没有许可证文件，原始快照缺少部分内部模块；后续提交补入 stub 和 1.0.0-dev 构建值。真正版本由外部 MACRO.VERSION 注入，因此不能把该快照直接标成 2.1.220。

使用原则：黑盒与源码一致的规则标为“相互印证”；只在镜像中出现的 MCP、Memory 或全屏分支标为“静态发现”，不冒充 2.1.220 默认行为。

研究记录: claude-code-tool-folding-rules-20260806.md; claude-code-source-tool-folding-tanbiralam-20260806.md。

Pi Stuff 当前实现

Pi Stuff 把分组当作成功结束后的显示优化，而不是执行中的主视图。它用保守分类器寻找同一 assistant message 内的相邻探索调用，只有整组成功才把后续行降为零高度。

候选类型	Read、Grep、Find、List；安全只读 Bash；Web retrieval；只读 Context；MCP 搜索、描述和状态。
硬边界	普通文本、非探索 Tool、assistant message 结束；候选少于 2 个不成组。
结果门槛	每个成员必须可见、可折叠且为 success；失败、拒绝、取消与后台移交使整组逐项显示。
重建路径	session start、reload、resume、tree 与 compaction 后从 Host context 重新计算。

Bash 为什么最保守

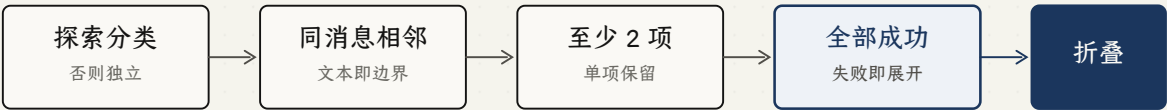
unbash 解析 AST 后才允许进入分组。命令长度上限为 32 KiB；后台 statement、shell command prefix、危险重定向、动态执行、写入式 git/bd/gh、测试和构建均被排除。分类异常时默认 standalone，而不是猜测为只读。

折叠后的投影

- Explore 4 operations · Read, Find +2 more

第一项成为组首；其余成员保留绑定但渲染为零高度。摘要最多显示两个不同 Tool 标签，重复标签使用 ×N，其余调用使用 +N more，耗时取成员最大值。

策略流程



每个门槛都 fail closed；展示摘要只在最后一步出现。

Pi Stuff 把折叠放在全部成功之后；任一不确定状态都会回到逐项显示。

Claude Code 2.1.220 行为

Claude Code 把折叠作为连续探索段的默认投影，从第一个调用开始就聚合。它跨 API message 维护语义计数，并在运行中显示最新目标，完成后只留下过去时统计。

最小门槛	1 个 retrieval operation；单次 Read 也折叠。
连续窗口	同一用户回合内可跨多个 assistant API message；assistant 正文或非 retrieval Tool 切断。
语义类别	Reading、Searching for、Listing；Bash cat、grep、ls 按含义归类。
详情入口	Ctrl+O 切换全局详细 transcript，恢复每一个原始调用。

同一组的两个生命周期投影

运行中

● Searching for 2 patterns, reading 3 files...

└─ src/config.ts

完成后

Searched for 2 patterns, read 3 files

运行态用现在时并展示最近目标；结束后改为过去时并移除目标。重复读取同一路径按唯一文件数计数，搜索与列表按相应操作统计。

错误仍留在成功口吻中

实测 Read(a) → Read(missing) → Read(b) 最终显示 Read 3 files。失败调用仍被计数并隐藏，只有进入详细 transcript 才能看到。这是两套方案最重要的产品分歧。

行为边界：默认 retrieval folding、单个 Tool 输出截断和可选 Focus view 是不同机制。本页只陈述 2.1.220 默认视图的黑盒结果。

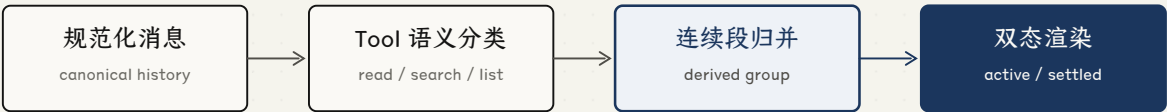
主证据：官方 Linux x64 2.1.220 隔离 PTY；辅助证据：Interactive mode、Fullscreen mode 与 CHANGELOG。

公开镜像揭示的实现链路

镜像中的折叠不是 Tool 执行器行为，而是一个**纯显示流水线**：Tool 声明语义，消息层扫描连续段并生成派生 group，渲染层根据 active 状态选择现在时、目标提示或完成态统计。

6f6f12 固定源码快照	489 MCP 搜索与读取白名单唯一名称	700 ms 最近目标最短显示时间
-------------------------	--------------------------------	-----------------------------

源码链路



Messages.tsx 生成显示投影；原始 normalizedMessages 继续用于 lookup 与 transcript。

Tool 分类、连续段归并和双态渲染相互独立；canonical message 与派生 display group 分开。

分类归属	Read、Grep、Glob 直接声明；Bash 与 PowerShell 单独解析；MCP 使用显式 allowlist；未知名称和 WebSearch/WebFetch 默认独立。
分组边界	正文、非折叠 Tool、非匹配 user result 切断；thinking、system 与多数 attachment 跳过或延后。
计数与错误	文件路径用 Set 去重；搜索、列表按调用；tool result 不检查 is_error。
运行稳定性	计数只增不减；最近目标至少显示 700 ms，避免快速 Tool 只闪一帧。

Bash 安全差异：镜像用 shell-quote 拆词，并跳过重定向操作符及目标；重定向本身不会否决 read/search/list 分类。ToolSearch、Snip 与自动管理的 Memory 另有静默或专用汇总规则。

条件路径：经典终端依赖 Ctrl+O 全局展开；源码中的全屏路径还支持行级点击或 Enter 展开，并可汇总普通 Bash 和 Git 结果。外部用户默认不启用该路径，不能把它写成 2.1.220 默认行为。

固定引用：collapseReadSearch.ts: 143-956；CollapsedReadSearchContent.tsx: 29-483；BashTool.tsx: 60-171；classifyForCollapse.ts: 14-604。

差异不是样式，而是风险模型

两套方案最明显的文字差异只是表层。决定用户是否信任摘要的，是候选窗口有多宽、摘要何时出现、失败是否可见，以及退出摘要后能否迅速恢复原始证据。

维度	Pi Stuff 当前	Claude 行为与镜像	用户或工程影响
最小规模	至少 2 项	1 项即折叠	Pi 保留偶发调用；Claude 密度更高
分组窗口	同一 assistant message	连续 retrieval segment，可跨 API message	Pi 容易被协议轮次切碎
分类职责	中央 exploration classifier	Tool-owned 接口加显示层扫描器	Claude 更易按 Tool 扩展，Pi 更集中审计
摘要与时机	Explore + Tool 标签；结束后聚合	语义计数；首项开始聚合	Claude 更安静，Pi 更容易出现收束跳动
运行目标	各行显示自己的目标	最近目标，至少稳定 700 ms	Claude 用一个方向提示替代多行噪声
完成目标	只留 Tool 标签	默认移除目标	两者都有回看成本，Claude 更明显
失败	整组逐项显示	仍折叠并计数	Pi 强调异常证据；Claude 强调连续性
Bash	unbash AST 与副作用否决	命令白名单；重定向被跳过；全屏可汇总全部 Bash	Pi 更保守；Claude 条件路径已超出 retrieval
外部 Tool	Web retrieval 可折叠；真实 MCP 调用独立	WebSearch/WebFetch 独立；MCP 显式 allowlist	两者的语义覆盖面并不等价
详情	/tools 局部详情	Ctrl+O 全局；全屏可行级展开	局部定位与完整上下文各有优势
持久化	JSONL 与模型结果不变	派生 group，原始消息保留	两者都具备可逆显示投影

各自付出的代价

Pi Stuff 的误折叠成本较低，但探索链条会被 API message 人为切碎，执行阶段仍出现多行，Explore + Tool labels 也没有直接说明读了什么。Claude Code 的阅读连续性更强，但单次调用也被隐藏，完成后没有路径线索，失败还会被成功口吻吞没。

源码说明 Claude 的优势来自职责拆分，而不是一个更宽松的正则。Pi Stuff 可以借 Tool 声明、显示 accumulator 与 active renderer 的分层，不必接受失败计数或普通 Bash 汇总。

SHARED INVARIANT

两者都把 compact view 视为持久化调用之上的派生投影。后续优化应继续限制在 presentation layer，不合并 Tool result，不删除 session entry，不改变模型上下文。

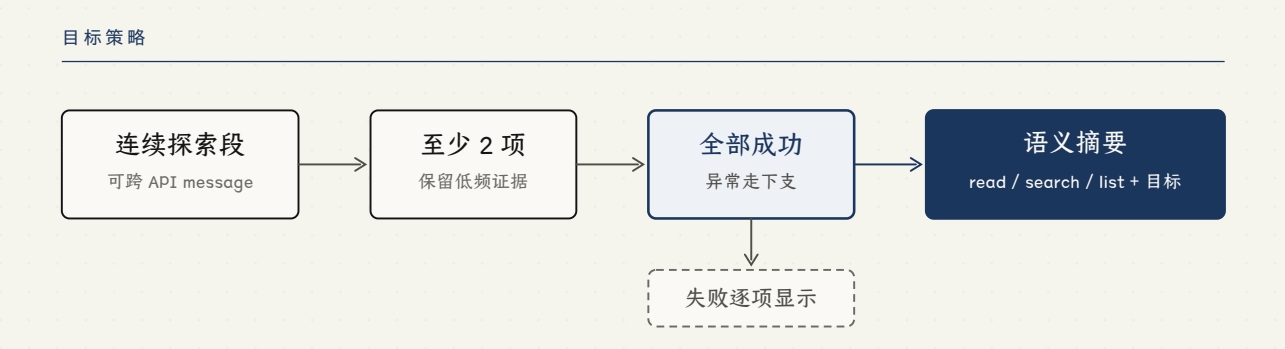
建议的演进路径

下一版不应复制 Claude Code, 而应组合它的信息架构与 Pi Stuff 的安全边界。优先改分组语义和运行态, 再扩大窗口; 错误可见性、最小两项门槛与完整详情入口继续作为产品约束。

决策	规则	原因
借鉴	read/search/list 语义摘要; 单调计数; 稳定的最近目标; 跨 message 连续探索段	降低 Tool 名噪声, 让摘要匹配用户对调查阶段的理解
保留	至少 2 项; 失败整组展开; AST Bash 判定; Web retrieval; /tools 与 JSONL 不变	维持异常证据、可逆性、协议稳定性和现有覆盖面
拒绝	单项折叠; 完成后删除全部目标; 失败写成 Read N files; 默认汇总普通 Bash	密度收益不足以抵消可追溯性与副作用风险

三阶段实施顺序

- 语义与运行态: 改成读、搜、列结果类别; 引入单调计数和 500 至 700 ms 目标稳定时间; 完成后至少保留短路径或共同根目录。
- 连续探索段: 跨 assistant message 维护 segment; 正文、写操作、普通 Bash、后台移交和用户输入仍是硬边界; 错误立即取消折叠资格。
- 分类职责: 再评估让各 Suite 声明稳定展示语义; MCP 使用审计过的 allowlist, 未知 Tool 一律 standalone, Bash 继续由 AST 判定。



目标策略吸收 Claude 的连续探索段、职责拆分和语义摘要, 同时保留 Pi Stuff 的失败展开与两项门槛。

验收条件

真实 PTY 必须继续覆盖 reload、resume、tree、compaction、失败展开、后台边界、目标稳定时间、窄终端、/tools 完整性和 JSONL 无展示摘要。任何一项失败, 都不应以更激进的默认折叠换取表面密度。

参考资料

- Pi Stuff cf12251; contract.ts; exploration.ts; PTY lifecycle certification
- docs/research/claude-code-tool-folding-rules-20260806.md
- docs/research/claude-code-source-tool-folding-tanbiralam-20260806.md; mirror 6f6f12
- Claude Code official Interactive mode、Fullscreen mode 与 CHANGELOG
- Anthropic issue #21151: 完成态折叠的可追溯性反馈